GUIDE 08 · QUERIES

Cache a saved query's response

The latest version is at anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?g=08-cache-a-saved-query. This copy was made on 4 November 2026.

Time: 5 minutes. **You'll end up with:** a saved query that skips the database entirely on a repeat call within its TTL - verified here with a real, deliberately stale result, so "cached" isn't just taken on faith.

Turn it on

At save time:

```shell
curl -X POST http://127.0.0.1:5000/api/v1/queries -H 'X-API-Key: demo-key' -H 'Content-Type:
application/json' -d '{
  "name": "hit_count", "sql": "SELECT COUNT(*) AS n FROM hits WHERE path = :path",
  "parameters": {"path": {"type": "str", "default": "/x"}},
  "connection_name": "shop", "author": "you", "description": "Hit count by path",
  "cache_ttl": 30,
  "publish": true
}'
```

`cache_ttl` is seconds, any non-negative integer, no upper bound. Retune or turn off an existing query's caching **in place** - not a new version, doesn't touch `execution_history` or the version number:

```shell
curl -X PATCH http://127.0.0.1:5000/api/v1/queries/hit_count/versions/1 -H 'X-API-Key: demo-key' \
  -H 'Content-Type: application/json' -d '{"cache_ttl": 5}'
curl -X PATCH http://127.0.0.1:5000/api/v1/queries/hit_count/versions/1 -H 'X-API-Key: demo-key' \
  -H 'Content-Type: application/json' -d '{"cache_ttl": 0}'   # 0 or null turns it off
```

(`1` is the version - each version has its own `cache_ttl`; the one callers get is the published one.)

Proof it's actually skipping the database, not just labeling the response

```shell
curl -i 'http://127.0.0.1:5000/q/hit_count' -H 'X-API-Key: demo-key'
# X-Cache: MISS
# ETag: "f22c7a09230338d15d1faa7812a0bf9f8d8d555c98cc6e1ea791f59deceabad0"
# Cache-Control: private, max-age=30
```

Now change the underlying data **directly**, bypassing the API entirely (something only caching, not any kind of request-level trickery, could hide):

```python
import sqlite3
c = sqlite3.connect('shop.db')
c.execute("INSERT INTO hits (path) VALUES ('/x')")
c.commit()
```

```shell
curl -i 'http://127.0.0.1:5000/q/hit_count' -H 'X-API-Key: demo-key'
# X-Cache: HIT
# [{"n": 1}]   ← the OLD count, verified stale on purpose - the row that was just inserted is real
```

That's the actual guarantee `cache_ttl` gives you: for up to 30 seconds, this exact call answers from memory, correct or not, and the database is never touched. Decide whether that's the right trade for a given query with that in mind - not "is this data important" but "is being up to `cache_ttl` seconds stale acceptable for this specific call."

Send the `ETag` back to skip the body entirely once you already have it cached client-side:

```shell
curl -o /dev/null -w '%{http_code}\n' 'http://127.0.0.1:5000/q/hit_count' -H 'X-API-Key: demo-key' \
  -H 'If-None-Match: "f22c7a09230338d15d1faa7812a0bf9f8d8d555c98cc6e1ea791f59deceabad0"'
# 304
```

What counts as "the same call" for caching purposes

A cache hit is scoped to the **exact** combination of name, version, connection, resolved parameter values, output `format` and page - anything else is a separate entry, a separate MISS:

```shell
curl -i 'http://127.0.0.1:5000/q/hit_count?path=/y' -H 'X-API-Key: demo-key'
# X-Cache: MISS - different parameter value, unrelated to the /x entry above
```

A cache hit is also never recorded in `execution_history` - nothing ran against the database, so there's nothing to log a run for.

What's never cached, no matter what you set

`cache_ttl` on a query whose SQL is a write (`INSERT` / `UPDATE` / `DELETE` /DDL) is silently ignored - verified: setting `cache_ttl: 30` on an `INSERT` and calling it twice ran two real inserts, no `X-Cache` header at all, ever. This is deliberate, not a bug to route around: caching a write would mean every call after the first silently stops writing, which is a much worse failure mode than "caching didn't help this one." Mongo saved queries can't be cached yet either (see Connect to MongoDB), and neither can a streamed (`?stream=true`) export - caching would mean materializing the whole body anyway, defeating the point.

Where cached responses actually live

By default, in this one server process's memory - **gone on restart**, verified: a cached entry from before a restart is simply not there afterward, a fresh `MISS` on the next call. Set `QUERYAPIGATE_REDIS_URL` to share the cache across restarts, or across several QueryAPIGate instances, via Redis instead - same `X-Cache` / `ETag` behavior either way, just a different backend. Either way, the cache is shared across every API key that can reach the connection - it stores nothing an authorized caller couldn't already see by running the query itself, so there's no per-key isolation to think about.

Browsing and clearing the cache

```shell
curl http://127.0.0.1:5000/api/v1/cache/entries -H 'X-API-Key: demo-key'
# {"items": [{"key": "...", "meta": {"name": "hit_count", "version": 1, ...}, "size_bytes": 10,
"ttl_remaining_s": 25.7}]}

curl -X DELETE http://127.0.0.1:5000/api/v1/cache/entries/<key> -H 'X-API-Key: demo-key'  # evict one
early
curl -X DELETE http://127.0.0.1:5000/api/v1/cache/entries -H 'X-API-Key: demo-key'        # clear
everything
```

The admin UI's **Caching** tab (under **Data**) shows the same thing - every live entry, what's cached in it (the exact response body, if you want to inspect it), and how long until it expires - plus the current backend (in-process or Redis) under **Settings**.

When it's actually worth it

Good fit: a query that's expensive or slow relative to how often the underlying data really changes - a dashboard KPI polled every few seconds where "accurate to the last 30 seconds" is completely fine (this is exactly what the built-in example APIs' dashboard scenario does - every KPI query there carries `cache_ttl=30`). Bad fit: anything where a caller needs to see the effect of its own or someone else's write immediately after it happens - caching would mask that for up to the whole TTL.

Next steps

- Group queries into a collection - the dashboard-scenario pattern (several cheap, cached, frequently-polled queries under one collection) is worth combining with this.
- Wire up Prometheus and Grafana - `queryapigate_cache_hits_total` / `queryapigate_cache_misses_total` show up in `/metrics` either way (in-process or Redis-backed), worth watching once caching is live.